Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información
y Ciencias de la Computación

ER-AFD / DOCUMENTO 01 DE 03

Reporte técnico

Convertidor de expresiones regulares a AFD

Construcción explícita ER → AFND → AFD

PRESENTADO POR

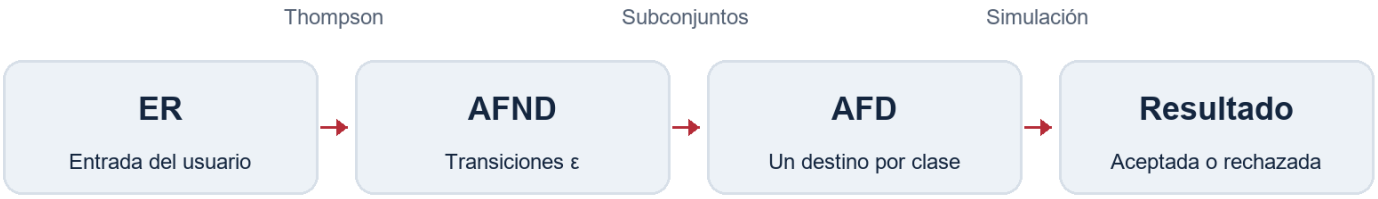
SELVIN KLEBER GIRÓN ALVAREZ	**Carné: 3190-15-15973**
JACKELIN PAULINA AVILA CIFUENTES	**Carné: 6590-24-772**
JOSE JULIO JIMENEZ MATUL	**Carné: 6590-24-13855**
MARCOS RODOLFO ARANA APOLINAR	**Carné: 1490-22-16405**

Guatemala, 20 de octubre de 2026

Proyecto del curso · Implementación en Python

1. Propósito y estructura

El proyecto implementa una herramienta de línea de comandos que recibe una expresión regular y una cadena ASCII. Construye explícitamente un autómata finito no determinista (AFND) con transiciones épsilon y después su autómata finito determinista (AFD), para decidir si la cadena pertenece al lenguaje descrito.



Objetivos implementados

Construir el AFND mediante Thompson; obtener el AFD por construcción de subconjuntos; mostrar estados, alfabeto, estado inicial, estados finales y transiciones; y evaluar la cadena mostrando cada paso del recorrido.

Archivo	Responsabilidad principal
expresiones.py	Análisis sintáctico y construcción del AFND.
automatas.py	Partición del alfabeto, cierre ε, construcción del AFD y evaluación.
er_afd.py	Programa principal, argumentos, validaciones y presentación de resultados.
er-afd	Lanzador de consola que llama al mismo programa principal.
Makefile	Preparación, ejecución y limpieza en Linux.
pruebas.py	Verificación automatizada con la biblioteca estándar unittest.

La ejecución concreta admite una ER y una cadena por llamada. La versión documentada no incorpora lectura de archivos ni tokenización simultánea de varias reglas.

2. Análisis de la expresión

El analizador descendente recursivo lee la ER de izquierda a derecha. La precedencia, de menor a mayor, es unión, concatenación y cerradura. Los paréntesis permiten agrupar una subexpresión.

```
unión         := concatenación ("|" concatenación)*
concatenación := repetición repetición*
repetición    := átomo "*"*
átomo         := literal | ε | . | conjunto | "(" unión ")"
```

Los fragmentos de Thompson se construyen mientras se analiza la entrada. No se crea un árbol sintáctico adicional ni se utiliza una biblioteca de expresiones regulares para reconocer cadenas.

Entrada	Interpretación
ab / a|b / a* / (r)	Concatenación, unión, cerradura y agrupación.
. / ε	Cualquier carácter ASCII / cadena vacía, sin consumir entrada.
[abc] / [^abc]	Conjunto positivo / complemento respecto a los 128 caracteres ASCII.
[a-z]	Rango inclusivo y ascendente dentro de un conjunto.
* / \p / \\	Escapes de literales: asterisco, letra p y barra invertida.
\N	Dígitos del 0 al 9.
\hHH	Carácter por dos dígitos hexadecimales; códigos 00 a 7F.
\A / \a / \z	Letras mayúsculas / minúsculas / ambas.

Los conjuntos positivos y las clases especiales se expanden a caracteres individuales en el orden indicado. Los conjuntos negativos conservan una etiqueta de complemento. Se validan paréntesis, conjuntos, escapes, extremos de rangos y caracteres no ASCII.

3. Construcción de Thompson

Cada subexpresión produce un fragmento representado por su estado inicial y final. Las transiciones se almacenan en una lista de diccionarios: cada estado tiene etiquetas que apuntan a conjuntos de estados destino. Una etiqueta **None** representa ε.

Operación	Construcción aplicada
Literal o clase	Dos estados unidos por una transición consumible. Una clase positiva puede tener varias etiquetas entre esos mismos estados.
Épsilon	Dos estados unidos por una transición ε.
Concatenación rs	Conectar el final de r con el inicio de s mediante ε.
Unión r|s	Agregar inicio y final nuevos; conectar ambas alternativas mediante ε.
Cerradura r*	Agregar inicio y final nuevos; permitir omitir r, entrar, repetir y salir mediante ε.

Ejemplo: a|b

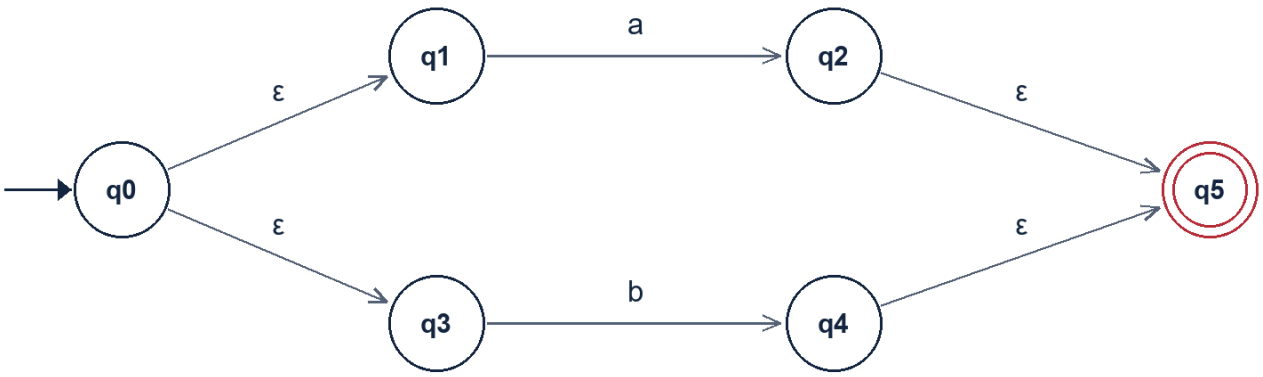


Figura 1. AFND de la ER a|b. La flecha de entrada marca q0 y el círculo doble marca q5 como final.

El inicio de Thompson puede crearse después de otros estados. Al terminar se renumeran todos los estados para que el inicio sea **q0**, conservando las conexiones. El alfabeto mantiene el orden de aparición y ε se agrega al final si no apareció explícitamente en la ER.

4. Construcción del AFD

Cierre épsilon

El cierre ε de un conjunto S contiene sus propios estados y todos los alcanzables siguiendo solamente transiciones ε. Se usa una lista de pendientes y un conjunto de visitados para no repetir estados, incluso cuando existen ciclos.

Construcción de subconjuntos

```
inicio = cierre_ε({q0})
registrar inicio como Q0
mientras existan subconjuntos pendientes:
    tomar un subconjunto S
    para cada clase de entrada C:
        T = cierre_ε(mover(S, C))
        registrar T si todavía no existe
        guardar la transición de S hacia T
    marcar S como final si contiene el final del AFND
```

Los estados del AFD se numeran en el orden de descubrimiento como Q0, Q1, Q2 y así sucesivamente. Si aparece el subconjunto vacío, se conserva como estado sumidero: sus transiciones permanecen en él.

Punto y conjuntos negativos

Una etiqueta como punto puede coincidir con la misma letra que otra transición literal. Por eso se divide el dominio ASCII en clases disjuntas según las etiquetas que coinciden con cada carácter. Todos los caracteres de una clase comparten los mismos destinos.

Por ejemplo, en **.b|ac**, al leer a desde el inicio se suman los destinos del punto y del literal a. Esto permite aceptar tanto ab como ac. La tabla conserva Σ de la ER como referencia e informa las clases efectivas cuando difieren.

No se realiza minimización: distintos subconjuntos alcanzables se conservan como estados distintos, aunque en algunos casos reconozcan el mismo comportamiento.

5. Ejemplo completo: a|b

El AFND tiene seis estados, inicio q0 y final q5. Su alfabeto mostrado es {a,b,ε}. Las siguientes filas corresponden a las transiciones generadas por el programa.

Estado	Símbolo	Destinos AFND
q0	ε	{q1,q3}
q1	a	{q2}
q2	ε	{q5}
q3	b	{q4}
q4	ε	{q5}

Subconjuntos y matriz del AFD

Estado	Subconjunto AFND	Con a	Con b	Final
Q0	q0,q1,q3	Q1	Q2	No
Q1	q2,q5	Q3	Q3	Sí
Q2	q4,q5	Q3	Q3	Sí
Q3	∅	Q3	Q3	No

Q0 es el estado inicial. Q1 y Q2 son finales porque contienen q5; Q3 representa el subconjunto vacío. El alfabeto consumible del AFD es {a,b}.

```
Entrada: a
Q0|a|Q1
Cadena aceptada
```

```
Entrada: ab
Q0|a|Q1
Q1|b|Q3
Cadena rechazada
```

El ejemplo muestra que se evalúa la cadena completa. Haber pasado por un estado final no es suficiente si quedan caracteres pendientes.

6. Evaluación y decisiones técnicas

La evaluación empieza en Q0. Para cada carácter se localiza su clase y se consulta la fila de la matriz del estado actual. Se registra origen, carácter y destino. Al finalizar, la cadena se acepta si el estado actual pertenece al conjunto de finales.

Situación	Respuesta
Estado final después de consumir la cadena	Cadena aceptada; código de salida 1.
Estado no final o carácter fuera del alfabeto	Cadena rechazada; código de salida 0. Si falta transición, el recorrido muestra ∅.
ER inválida, entrada no ASCII o anidamiento excesivo	Mensaje de error; código de salida 2.

Costo y límites

Si n es el número de estados del AFND, un AFD puede llegar a tener hasta 2 elevado a n subconjuntos. La implementación construye solamente los alcanzables y mantiene como máximo 128 clases ASCII. El cierre ε recorre estados y transiciones alcanzables; la simulación del AFD es lineal respecto a la longitud de la cadena, después de preparar el mapa de caracteres.

La versión usa únicamente la biblioteca estándar de Python, distingue mayúsculas de minúsculas y no interpreta operadores adicionales como + o ?. Los grupos y alternativas vacíos se rechazan; para el vacío debe escribirse ε. El anidamiento se limita por la recursión disponible y se informa con un error controlado.

Verificación y documentos relacionados

La batería terminó con 10 métodos correctos. Incluye 152 evaluaciones dirigidas de 42 ER, 24 ER inválidas, 1,023 combinaciones y 256 comprobaciones sobre complementos ASCII. Los resultados y las capturas se detallan en casos_de_prueba.pdf.

Las evidencias se obtuvieron en Windows 11 con Python 3.12.10. Los comandos del Makefile corresponden al entorno Linux solicitado, pero no se ejecutaron en Linux durante esta sesión. El repositorio remoto está pendiente de publicación.

Fuentes de esta documentación

Enunciado proporcionado: ER-AFD, Convertidor de ER a AFD, René Ornélyz, septiembre de 2026. Implementación local: expresiones.py, automatas.py, er_afd.py, er-afd, Makefile y pruebas.py. Evidencia reproducible: doc/evidencias/ejecuciones.json.